

EXPERIMENT MANUAL

CFT-TimeVAE for Water Demand Forecasting

Reproducible workflow for preprocessing, synthetic data generation, forecasting, and validation

Repository

github.com/Biniam150/CFT-TimeVAE-WaterDemandForecasting

Purpose: Follow the stages in order. The preprocessing outputs are shared by CFT-VAE and TimeVAE; each model then has its own synthetic-data and forecasting pipeline.

Version 1.0 | July 2026

1. Purpose and scope

This manual documents the execution order for reproducing the water-demand experiments in the repository. It covers data retrieval, quality-controlled input data, aggregation and temporal preprocessing, CFT-VAE and TimeVAE synthetic-data generation, six forecasting models, statistical analyses, computational profiling, and conditioning-mechanism ablation.

Run order: Complete Stage 1 once. Then run the CFT-VAE and TimeVAE branches independently. Analysis scripts should be run only after their required forecasting-result files exist.

1.1 Workflow at a glance

Stage	Action	Primary output
1	Aggregate, split, engineer, and normalize data	outputs/temporal_split_data/
2A	Train CFT-VAE and generate conditional synthetic data	outputs/CFT-VAE/
2B	Train TimeVAE and generate unconditional synthetic data	outputs/Time-VAE/original_timevae_results/
3A	Run CFT-VAE forecasting: four classical models plus LSTM/TCN	outputs/CFT-VAE/CFT-VAE-forecasting_results/
3B	Run TimeVAE forecasting: four classical models plus LSTM/TCN	outputs/Time-VAE/Time-vae_forecasting_results/
4	Run validation, robustness, scalability, and ablation analyses	outputs/Visualization/ and experiment folders

2. Prerequisites and installation

Use PowerShell from the repository root for the commands in this manual.

1. Install Git and Git LFS on the computer.
2. Configure Git LFS.

```
git lfs install
```

3. Clone the repository and retrieve the LFS-managed data files.

```
git clone https://github.com/Biniam150/CFT-TimeVAE-WaterDemandForecasting.git
cd CFT-TimeVAE-WaterDemandForecasting
git lfs pull
```

4. Install the project dependencies.

```
pip install -r requirements.txt
```

Working directory: Run every command from the repository root unless a section explicitly says otherwise.

3. Data inputs

3.1 Original smart-meter data

The original file is named `swm_trialA_1K.csv` and is stored at:

```
data/raw/swm_trialA_1K.csv
```

The source dataset is Household Water Consumption (Average), available from HARDMIN-HELIX:

```
https://hardmin.heal-link.gr/en/dataset/700cde6f-ef69-41f0-8fec-9c3f74ae38d4
```

The source contains hourly smart-water-meter time series for 1,007 consumers.

3.2 Quality-controlled data

The experiment input is the quality-controlled file:

```
data/Processed_data/Good_Data.csv
```

- The retained time series contain no missing time steps.
- Negative water-consumption values are excluded.
- The large CSV files are managed by Git LFS.

4. Stage 1 - Aggregation and temporal preprocessing

Run the preprocessing script:

```
python "src/Aggregation&Temporal_Preprocessing/Aggregation&Temporal_Preprocessing.py"
```

The script performs the following operations:

- Loads data/Processed_data/Good_Data.csv.
- Creates randomized user aggregations at the configured aggregation levels.
- Constructs the aggregated water-demand target and conditional temporal features.
- Applies temporal training, validation, and testing splits.
- Normalizes the resulting features and stores normalization metadata.
- Saves one NPZ file for each aggregation-level and randomized-trial combination.

```
outputs/temporal_split_data/
|-- train/train_preprocessed_level<level>_trial<trial>.npz
|-- val/val_preprocessed_level<level>_trial<trial>.npz
`-- test/test_preprocessed_level<level>_trial<trial>.npz
```

Completion check: Confirm that matching train, validation, and test files exist for every trial you intend to use.

5. Stage 2A - CFT-VAE synthetic data generation

CFT-VAE uses the architecture in src/CFT-VAE/Model_definition.py.

```
python "src/CFT-VAE/Trainning.py"
```

At the prompts, select an aggregation level, the number of randomized trials, and a supported synthetic-data ratio. The script trains one model per trial and generates conditional synthetic demand sequences.

```
outputs/CFT-VAE/
|-- history/history_level<level>_trial<trial>.csv
|-- metamodels/weights_level<level>_trial<trial>.h5
`-- synthetic_data/ratio<ratio>/synthetic_level<level>_trial<trial>_ratio<ratio>.npz
```

Dependency: Do not run CFT-VAE forecasting until the selected trials have corresponding synthetic NPZ files.

6. Stage 3A - CFT-VAE forecasting

Both forecasting scripts evaluate real-only, synthetic-only, and augmented training scenarios.

Group	Script	Models	Output file
Classical	Forecasting.py	Random Forest, XGBoost, Gradient Boosting, SVM	results_level<level>_ratio<ratio>.csv
Deep learning	forecasting_lstm_tcn.py	LSTM, TCN	results_level<level>_ratio<ratio>_lstm_tcn.csv

```
python "src/CFT-VAE/Forecasting.py"
python "src/CFT-VAE/forecasting_lstm_tcn.py"
```

Results are written to:

```
outputs/CFT-VAE/CFT-VAE-forecasting_results/
```

7. Stage 2B - TimeVAE synthetic data generation

The original TimeVAE baseline uses the architecture in src/Time-VAE/Time_vae_model.py.

```
python "src/Time-VAE/Train.py"
```

Select an aggregation level and the number of randomized trials. The script trains one original TimeVAE model per trial and generates unconditional synthetic sequences.

```
outputs/Time-VAE/original_timevae_results/
|-- models/
|-- training_histories/
|-- meta_histories/
`--
synthetic_data/ratio_<ratio>/synthetic_original_level<level>_trial<trial>_ratio<ratio>.npz
```

8. Stage 3B - TimeVAE forecasting

TimeVAE forecasting uses the same real-only, synthetic-only, and augmented scenarios.

Group	Script	Models	Output file
Classical	Forecasting.py	Random Forest, XGBoost, Gradient Boosting, SVM	results_level<level>_ratio<ratio>.csv
Deep learning	forecasting_lstm_tcn_timevae.py	LSTM, TCN	results_lvl<level>_lstm_tcn_original_timevae_ratio<ratio>.csv

```
python "src/Time-VAE/Forecasting.py"
python "src/Time-VAE/forecasting_lstm_tcn_timevae.py"
```

Results are written to:

```
outputs/Time-VAE/Time-vae_forecasting_results/
```

9. Analysis and visualization

Run an analysis only after its required forecasting-result files and synthetic datasets exist. The repository organizes analyses by paper section.

Analysis	Folder under src/Visualization	Purpose
Baseline forecasting	baseline forecasting/	Baseline forecast visualization
Direct fidelity	Direct Fidelity Assessment/	Direct statistical comparison of real and synthetic demand
Model comparison and validation	Synthetic Data Generation, Forecasting, and Statistical Validation Results/	CFT-VAE/TimeVAE comparison and paired significance tests
Robustness	Robustness Analysis of CFT-VAE Augmentation Performance/	Augmentation success and robustness
Gain distribution	Distributional Analysis of Augmentation Gains/	Distribution of improvements across trials
Scalability	Computational Performance and Scalability Analysis/	Training and six-model forecasting profiles

Analysis	Folder under src/Visulization	Purpose
Conditioning ablation	Ablation Analysis of the Conditioning Mechanism/	Synthetic generation and strong-trial forecasting by mask

10. Conditioning-mechanism ablation

10.1 Generate CFT-VAE ablation datasets

```
python "src/Visulization/Ablation Analysis of the Conditioning Mechanism/Synthetic Data Generation/Train.py"
```

Run the script separately for each conditioning configuration:

Output folder	Conditioning configuration
mask_[]	Full conditioning
mask_[1]	No lag features
mask_[2]	No rolling and change features
mask_[3]	No calendar features
mask_[4]	No daily-context features
mask_[5]	No stability features
mask_[1, 2, 3, 4, 5]	No conditioning

10.2 Generate classical augmentation baselines

```
python "src/Visulization/Ablation Analysis of the Conditioning Mechanism/Synthetic Data Generation/Gaussian.py"
```

- Gaussian noise injection
- Time warping
- Window slicing

10.3 Run strong-trial forecasting

```
python "src/Visulization/Ablation Analysis of the Conditioning Mechanism/Forecasting/strong_trial_ablation_forecasting.py"
```

Input requirement: The strong-trial script requires the main CFT-VAE classical and LSTM/TCN reference-result CSV files plus synthetic files for every selected condition.

11. Reproducibility checks

- Use matching aggregation level, trial number, and synthetic ratio across training and forecasting.
- Do not rename NPZ or CSV files; downstream scripts discover files from their naming patterns.
- Retain normalization metadata stored in the processed NPZ files.
- Record the selected level, number of trials, ratio, and script version for every run.
- Run classical and LSTM/TCN forecasting scripts before statistical comparison.
- Keep generated outputs outside src unless a paper figure or table is intentionally copied into its documentation folder.

12. Troubleshooting

Symptom	Likely cause	Action
Only an LFS pointer is visible	Git LFS objects were not downloaded	Run git lfs install, then git lfs pull.
No trials are discovered	Expected preprocessing files are missing or misnamed	Check outputs/temporal_split_data/train and matching level/trial names.
Synthetic file missing	Training was not run for the selected level, trial, or ratio	Generate the matching synthetic NPZ before forecasting.
Forecasting output is empty	Real, test, or synthetic inputs did not match	Verify the level, trial, ratio, and output directory.
Ablation validation fails	One or more masks or baseline CSVs are absent	Generate every selected condition and both reference result files.
Import error	Dependencies or the active environment are incomplete	Activate the intended environment and install requirements.txt.

Appendix A - Command quick reference

```
python "src/Aggregation&Temporal_Preprocessing/Aggregation&Temporal_Preprocessing.py"

python "src/CFT-VAE/Training.py"
python "src/CFT-VAE/Forecasting.py"
python "src/CFT-VAE/forecasting_lstm_tcn.py"

python "src/Time-VAE/Train.py"
python "src/Time-VAE/Forecasting.py"
python "src/Time-VAE/forecasting_lstm_tcn_timevae.py"
```